Universidade Cruzeiro do Sul
Ciência da Computação

Aula 5 – Introdução à Semântica das Linguagens de Programação

Disciplina: Paradigmas de Linguagens de Programação
Professor: Álvaro Colombero Prado

Objetivos



- Definir para que servem os nomes em uma linguagem de programação.
- Identificar e explicar as seis características relacionadas a uma variável.
- Diferenciar análise semântica estática e dinâmica.

Semântica

- Sintaxe é a forma da escrita.
  - Enquanto a descrição de sintaxe pode ser considerada algo simples, com uma notação universalmente aceita (BNF), isso não ocorre com a semântica.
- Semântica é um conjunto de significados.
  - Descrever semântica é uma tarefa difícil. Não há uma notação universalmente aceita para a representação de semântica. Muitas linguagens explicam sua semântica de formas diferentes.

Semântica

- Formas para especificação semântica:
  - Informal:
    - Descrição em linguagem natural das regras.
  - Semiformal:
    - Esquemas de tradução dirigidas pela sintaxe.
  - Formal:
    - Semântica operacional.
    - Semântica denotacional.
    - Semântica axiomática.

Na aula passada...



- Uma variável é uma abstração de uma ou mais células de memória identificada por um nome.
- Implemente o programa abaixo em C, execute o programa três vezes, anote e compare os resultados apresentados na tela.

```
#include <stdio.h>

int main(void) {

    int x=2147483647;

    printf("%d \n", x);

    printf("%ld \n", &x);

    return 0;
}
```

Nomes e Tipos de Dados

- Nomes são utilizados para identificar variáveis.
- Uma variável é uma abstração de uma ou mais células de memória que possui:
  - Nome (regras sintáticas).
  - Endereço (l-value).
  - Valor (r-value).
  - Tipo.
  - Escopo.
  - Tempo de vida.

Nomes e Tipos de Dados

- Endereço de uma variável (l-value)?
  - Indica o local da memória da máquina ao qual a variável está associada.
  - Um mesmo identificador pode representar diferentes posições de memória em diferentes partes da execução.
  - Diferentes variáveis podem indicar o mesmo endereço de memória.
- Valor de uma variável (r-value)?
  - Conteúdo da(s) célula(s) de memória associadas à variável.

Nomes e Tipos de Dados

- Tipo de uma variável:
  - Define a faixa de valores que a variável pode armazenar, o número de células de memória que ela utiliza e o conjunto de operadores que podem ser aplicados sobre ela.

Nomes e Tipos de Dados

- Escopo é o conjunto de comandos que podem acessar uma variável.
  - Uma variável é local a um escopo (conjunto de comandos) se ela for declarada dentro dele;
  - Uma variável é não local a um escopo se ela for acessível aos comandos, mesmo não sendo declarada dentro dele.
  - Em linguagens de programação, o escopo das variáveis pode ser **estático** ou **dinâmico**.

Nomes e Tipos de Dados

- Escopo estático:
  - O escopo de uma variável é definido estaticamente, conforme a posição de sua declaração no código.
  - Algumas linguagens exigem que a declaração seja feita no início dos módulos e outras, como C++, Java e C# não.
  - Divide-se em escopo global e local.
  - Uma variável é global quando é declarada fora de qualquer função, sendo acessível a todas elas.
  - Seu valor é acessível a todas as funções, exceto aquelas que declaram uma variável local com o mesmo nome.

Nomes e Tipos de Dados

- Escopo estático :
  - A declaração é buscada localmente (bloco ou função). Caso não seja encontrada, será buscada em seu ancestral, até o bloco de código mais abrangente. Caso não seja encontrada em nenhum deles, um erro é gerado.
  - A busca nos ancestrais pode gerar confusão em linguagens que permitem funções aninhadas.
  - Em C++ pode-se utilizar o operador de escopo :: para acessar variáveis globais em funções com variáveis locais homônimas.

Nomes e Tipos de Dados

- Escopo dinâmico:
  - A definição do escopo é feita em função da ordem de chamada dos subprogramas, o que pode variar entre uma execução e outra.
  - APL e as primeiras versões de LISP utilizavam escopo dinâmico.
  - Perl e Common LISP permitem a declaração de variáveis com escopo dinâmico, embora o padrão seja estático.
  - Os programas com escopo dinâmico são menos confiáveis e mais difíceis de serem lidos.

Nomes e Tipos de Dados

- Tempo de vida de uma variável:
  - O tempo de vida de uma variável pode ser definido como o tempo de sua existência em memória.
  - De maneira geral o escopo e o tempo de vida de uma variável estão relacionados.
  - A chamada a funções modifica esta relação.
  - Modificadores em C, C++ e PHP, como a palavra **static**, também alteram esta relação.

Nomes e Tipos de Dados

- Constantes nomeadas:
  - É um nome vinculado a um valor apenas uma vez.
  - Possibilitam a parametrização do programa, facilitando a sua manutenção e melhorando sua legibilidade e confiabilidade.
  - Algumas linguagens (Pascal e Fortran) permitem apenas a utilização de valores constantes, enquanto outras como C++ e Ada permitem expressões.

Nomes e Tipos de Dados

- Constantes nomeadas:
  - Em diferentes linguagens:
    - C/C++ - const.
    - Pascal – const.
    - Java – final.
    - C# - const e readonly.

Nomes e Tipos de Dados

```
import java.util.Scanner;
class Main {
  static final float pi = 3.1416f;

  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    System.out.println("Digite o raio de uma circunferência: ");
    float raio = Float.valueOf(input.nextLine());
    float area = pi*raio*raio;
    float comprimento = 2*pi*raio;
    System.out.println("Area: "+area);
    System.out.println("Comprimento: "+comprimento);
  }
}
```

Semântica Estática e Dinâmica

- A análise da semântica pode ser realizada em dois momentos. Por isto ela é dividida em semântica:
  - **Estática**: aquilo que pode ser verificado em tempo de compilação.

    Obs.: por definição, não há verificação estática em linguagens interpretadas!
  - **Dinâmica**: aquilo que só pode ser verificado em tempo de execução.

Bibliografia

- Material complementar:

TUCKER, A. B.; NOONAN, R. E.; **Linguagens de programação: princípios e paradigmas.** 2. ed. São Paulo: McGrall-Hill, 2009. Seções 4.3.

SEBESTA, R. W.; **Conceitos de linguagens de programação**. 9. ed. Porto Alegre: Bookman, 2011. Capítulo 5.